AML S26 Final Project

Jordan Miller and Sydney Williams

May 2026

Contents

0 Introduction and Acknowledgments **3**

1 Part I **3**
1.1 Data Overview . . . 3
1.1.1 File size, class balance, scale of features . . . 3
1.2 Design Decisions . . . 3
1.2.1 Pipeline elements (Traditional Method) . . . 3
1.2.2 Pipeline elements (Deep Learning Method) . . . 4
1.2.3 Model selection (Traditional) . . . 4
1.2.4 Model selection (Deep Learning) . . . 4
1.3 Traditional Results . . . 5
1.3.1 Development and training time . . . 5
1.3.2 Numerical metrics . . . 5
1.3.3 Confusion matrix . . . 6
1.3.4 Predicted results on test set . . . 6
1.4 Deep Results . . . 6
1.4.1 Development and Training Time . . . 6
1.4.2 Numerical metrics . . . 6
1.4.3 Confusion matrix . . . 7
1.4.4 Predicted results on test set . . . 7
1.5 Conclusion . . . 7
1.5.1 Model comparison . . . 7
1.5.2 Lessons learned . . . 8

2 Part II **8**
2.1 Data Overview . . . 8
2.1.1 File Size, Class Balance, Scale of Features . . . 8
2.2 Design Decisions . . . 8
2.2.1 Pipeline Elements (Traditional Method) . . . 8
2.2.2 Pipeline Elements (Deep Learning Method) . . . 9
2.2.3 Model Selection (Traditional) . . . 9
2.2.4 Model Selection (Deep Learning) . . . 10
2.3 Results . . . 11
2.3.1 Development and Training Time . . . 11
2.3.2 Numerical Metrics . . . 11
2.3.3 Confusion Matrix . . . 11
2.3.4 Predicted Results on Test Set . . . 13
2.4 Conclusion . . . 13
2.4.1 Model Comparison . . . 13
2.4.2 Lessons Learned . . . 15

0 Introduction and Acknowledgments

The goal of this project for ECE:5995 Applied Machine Learning is to combine all knowledge acquired over the course of the semester into a comprehensive machine learning solution to two problems. The steps followed were:

- Develop a suitable classification approach for each problem utilizing a a) traditional and b) deep learning method. Use appropriate preprocessing steps as needed.
- Optimize and compare the performance of both methods based on suitable performance metrics.
- Analyze and discuss your results as well as the time required for development and training of the classifiers.

LLM AI models were used for some portions of code development, specifically formatting, error checking, and creating result summary/plotting scripts. All final design decisions, including models and preprocessing used, as well as the report were made by Jordan and Sydney.

1 Part I

1.1 Data Overview

1.1.1 File size, class balance, scale of features

The PartI_dev.csv file contains 48 features. The dataset comprises 46,805 numerical samples evenly distributed across 11 distinct classes, with exactly 4,255 samples per class. This represents a perfectly balanced dataset, eliminating the need for class weighting or resampling methods. There are no missing values.

1.2 Design Decisions

The large number of samples and perfectly balanced classes allowed for a straightforward evaluation of various models without the need for complex resampling techniques.

1.2.1 Pipeline elements (Traditional Method)

StandardScaler was utilized to normalize the features to zero-mean and unit-variance. Normalization ensures that models sensitive to feature scale, such as K-Nearest Neighbors and Logistic Regression, perform optimally.

For dimensionality reduction, Principal Component Analysis (PCA) was evaluated. A variance threshold of 95% was applied to reduce the dimensionality of the feature space. With PCA, the dimensionality was reduced from 48 predictors to 31 predictors. Both the original raw features and the PCA-transformed features were tested to determine the most predictive input space.

1.2.2 Pipeline elements (Deep Learning Method)

Similar to the traditional approach, StandardScaler was used to normalize the raw features. Since the dataset is perfectly balanced, techniques like SMOTE or class-weighted loss functions were unnecessary. The deep learning model was trained directly on the normalized raw features.

1.2.3 Model selection (Traditional)

A variety of traditional machine learning models were benchmarked, including Logistic Regression, Random Forest, Gradient Boosting, K-Nearest Neighbors (KNN), Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), and Support Vector Machine (SVM).

In order to tune the hyper parameters for some models, GridSearchCV was used for KNN and Logistic Regression. K values from 1 to 50 were tested for KNN, and 0.01, 0.1, 1, and 10 were tested as values for Regularization Strength in Logistic Regression.

While GridSearchCV could have been used to tune hyperparameters in other tested models, the time it takes to train multiple models is not worth the small improvement. Many of the chosen hyperparameters are standard for balanced classes.

To ensure robust performance estimates, StratifiedKFold cross-validation with 5 splits was used to compare the models on the raw features. The models were evaluated on the mean accuracy score after 5 fold CV. Macro F1 score was also reported. Accuracy was chosen due to the perfectly balanced classes.

1.2.4 Model selection (Deep Learning)

A Multi-Layer Perceptron was selected for the deep learning approach. The network architecture consists of two hidden layers with 128 and 64 neurons, respectively. BatchNorm was included after each linear layer to normalize activations and stabilize training. ReLU was utilized as the activation function, along with Dropout(0.3) to prevent overfitting.

The model was compiled using CrossEntropyLoss, and optimized using Adam with a weight decay of 1e-4. Model performance was validated using 5-fold StratifiedKFold cross-validation.

1.3 Traditional Results

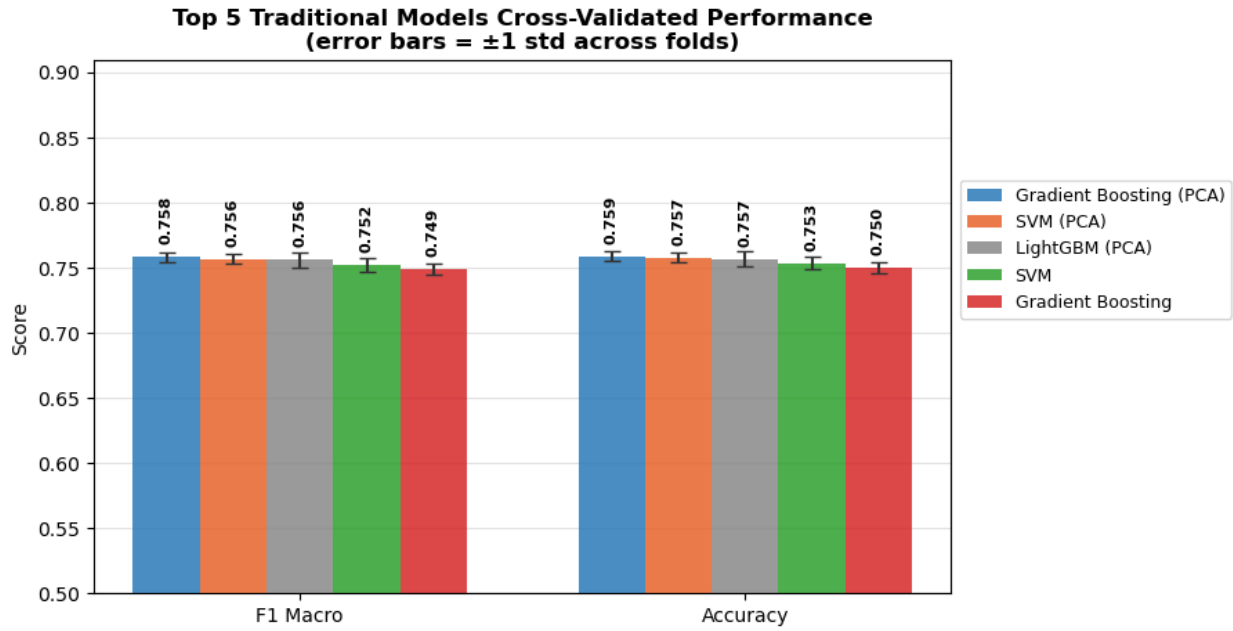

Figure 1: F1 Macro and Accuracy of Top 5 Traditional Models

1.3.1 Development and training time

Based on the CV results (Figure 1), Gradient Boosting + PCA was chosen as the best model for traditional machine learning. Gradient Boosting had the best mean accuracy, as well as the top F1 macro score.

After selecting Gradient Boosting + PCA as the best model for the given data set, it was retrained on the whole dataset. The machine used to train it was my laptop. With an AMD Ryzen 7 running at 3.30 GHz, re-training the model took 15.36 seconds.

While Gradient Boosting was chosen as the best model, SVM + PCA and LightGBM + PCA performed very similarly. LightGBM was about three times quicker to run 5-fold CV on than Gradient Boosting, so if we had a very large data set, the time save may be worth it. But in this case, 13 seconds versus 39 seconds is not a big difference, so the very small performance gain in Gradient Boosting is worth it.

1.3.2 Numerical metrics

Figure 1 shows a detailed report of all the models trained with 5-fold CV. Gradient Boosting + PCA performed the best with a mean accuracy score of 0.759, it had a macro F1 score of 0.758. Both SVM + PCA and LightGBM + PCA had an accuracy score of 0.756 and a F1 macro score of 0.757. While it cannot be seen in Figure 1, Quadratic Discriminant Analysis performed the worst with a mean accuracy score of 0.635 and a mean F1 macro score of 0.605.

After the best model was selected via 5-fold CV, it was tested on the 20% of data held out from the whole dataset. The model scored 0.76 for both F1 macro and accuracy.

1.3.3 Confusion matrix

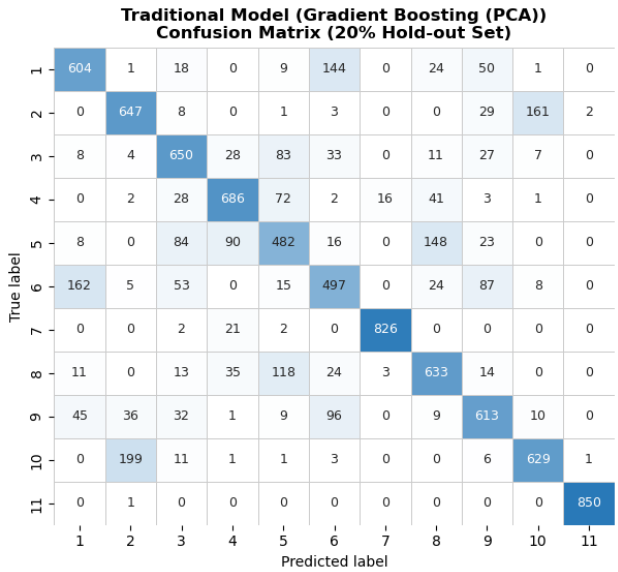

Figure 2: Confusion Matrix for GB + PCA on 20% holdout set

Figure 2 shows the confusion matrix on the 20% of data left out for testing, before it was trained on the whole dataset. There were a few common mis-classifications, with the most common being classifying class 10 as class 2. This mis-classification happened 199 times.

1.3.4 Predicted results on test set

Given that the test set is balanced like the development set, I predict that our model will perform similar to the development set. If classes that were easily confused (2,10 or 1,6) are more distinguished in the test set, I think it may perform slightly better.

1.4 Deep Results

1.4.1 Development and Training Time

The machine used to train the network used an NVIDIA A4500, with CUDA acceleration. The training time for the network on the full dataset after running CV was 167.44 seconds.

1.4.2 Numerical metrics

After using 5-fold CV (200 epochs/fold) to tune parameters, the network was able to achieve an average accuracy score of 0.761 and a F1 macro score of 0.759. After CV was finished, the network was tested on 20% of the data that was held out from the development set. On this dataset, the network achieved an accuracy score of 0.771 and a F1 score of 0.7689.

1.4.3 Confusion matrix

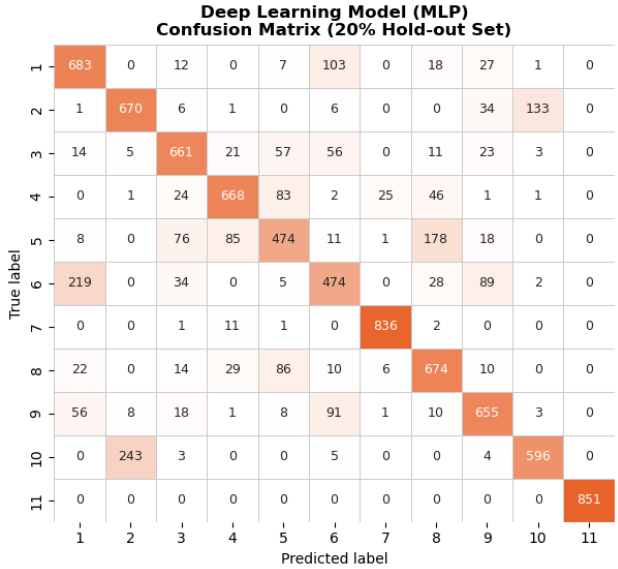

Figure 3: Confusion Matrix on held out data

Figure 3 shows the confusion matrix for the network. Comparing it to the confusion matrix from the traditional model, we can see that the same classes get mixed up. Classes 2 and 10 are often mistaken for each other, as well as classes 1 and 6.

1.4.4 Predicted results on test set

I think that the network will perform similarly on the test set. Again, if there is a better distinction between the commonly confused classes, I believe it could score a bit better on the test set. Deep learning models are often more flexible than traditional models, so it's likely that the deep learning model will perform slightly better than the traditional model.

1.5 Conclusion

1.5.1 Model comparison

The difference in accuracy and F1 scores on the held out data between traditional and deep learning is very minimal. Both of the held out datasets were selected using the same random seed, ensuring that they both saw the same held out data.

The big difference is the training time. The traditional model was re-trained on the whole dataset 10 times quicker than the deep learning model. The deep learning model was trained with a high end GPU as well, if there was no access to a GPU, the time difference would be much worse.

Given these factors, the traditional model is a slightly better fit for this dataset. While the performance is very marginally worse, the trade off between the tiny performance boost and the compute power/time needed to train the deep learning network is not worth it.

However, if the data set becomes bigger or more complex, the deep learning would become more appealing.

1.5.2 Lessons learned

Developing both the deep learning and traditional models presented their own challenges. Figuring out a good trade off between layer count and compute time was something I took away from the deep learning. The extra layer only provided a small gain, while greatly increasing the compute time. Similar to the deep learning model, I found out when it's worth trying to squeak out marginal gains on a model, and when it's not worth it due to compute time. The importance of PCA also stuck out to me. I think running PCA on the dataset was the step that improved model performance the most, while taking little time and compute power

Notably, both models struggled with the same misclassifications, particularly classes 2 and 10, and classes 1 and 6. This consistency suggests these classes are overlapping in the feature space rather than a limitation of either model architecture. Sometimes, the data is a limiting factor, not the model architecture.

2 Part II

2.1 Data Overview

2.1.1 File Size, Class Balance, Scale of Features

The PartII_dev.csv file contains 125 features. Each feature has 158 numerical samples, with 126 belonging to class 0 and 32 belonging to class 1. This represents a 4:1 class imbalance. The features have widely ranging scales and many feature pairs are correlated (—r—¿0.90). There are no missing values.

2.2 Design Decisions

The massive redundancy in features and very small sample size were the main constraints for this part of the project. Four iterations were tested with the final being the selected iteration of the models. Resampling and minority class synthesis techniques were experimented with, but with the dataset being so small they did not prove to be useful. The dataset being so small also caused consistent overfitting issues, meaning design decisions had to be carefully selected to minimize overfitting.

2.2.1 Pipeline Elements (Traditional Method)

StandardScalar was used in all versions of the traditional ML solution. StandardScalar normalizes the data to zero-mean and unit-variance, which was necessary due to the variety of scale in the dataset.

For dimensionality reduction, initially PCA (Principal Component Analysis) was selected. PCA discards all features except for the ones that account for 95% of the variance (in this case, all but 11 features). However, those high-variance features were not actually the most predictive features. Instead, SelectKBest was used. SelectKBest runs an ANOVA F-test on each feature independently against Y. The 10 most statistically discriminative features were kept and the rest were discarded.

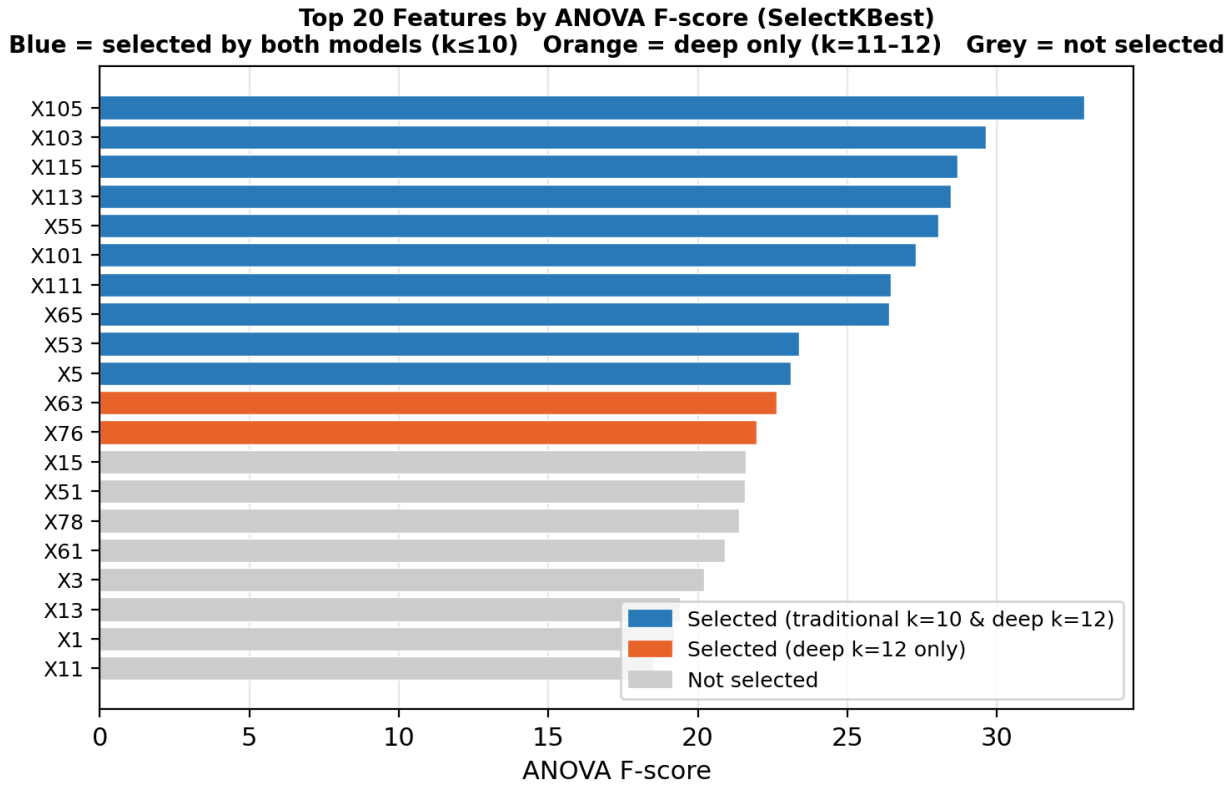

Figure 4: Comparison of features selected using SelectKBest for both Part II models

In earlier versions, SMOTE was used to generate synthetic minority-class samples. However, it did not show significant improvements in performance. LogReg can be set to class_weight(balanced) to adjust for the unbalanced classes instead.

2.2.2 Pipeline Elements (Deep Learning Method)

SelectKBest and StandardScalar are again used for the deep learning model. Like the traditional method, SMOTE was originally used. Not only did it not improve performance, it seemed to actively degrade it. In the final version there is no minority class synthesis.

2.2.3 Model Selection (Traditional)

To select a model, StratifiedKFold cross validation (specdifically RepeatedStratifiedKFold) with 5 splits repeated 5 times was used to compare models. CV F1, CV AUC-ROC,

and threshold were compared between LogReg with SelectKBest(k=10), LogReg with SelectKBest(k=7), SVM using RBF kernel and SelectKBest(k=8), and LogReg with PCA.

Model	CV F1 (macro)	CV AUC-ROC	Threshold
LogReg + SelectKBest(k=10)	**0.724 ± 0.066**	**0.738**	0.626
LogReg + SelectKBest(k=7)	0.724 ± 0.071	0.738	0.617
SVM(RBF) + SelectKBest(k=8)	0.702 ± 0.070	0.701	0.298
LogReg + PCA(95%)	0.648 ± 0.064	0.618	0.612

Figure 5: Result of model selection StratifiedKFold cross validation

Based on the CV results, LogReg with SelectKBest(k=10) was selected as the traditional learning method.

2.2.4 Model Selection (Deep Learning)

MLP (Multi-Layer Perceptron) was, based on research, used for this dataset. MLP is a deep learning model well-suited for non-linear class boundaries. After experimenting with model size, it was determined to be best to use a smaller model. The original large model not only took too long to train, but was also not performant on the very small dataset.

The final model used a single hidden layer. BatchNorm was used to normalize activations to zero mean/unit variance per batch, which stabilized training and acted as a mild regularizer. BatchNorm also reduced sensitivity to weight initialization. ReLU was used as the activation function with Dropout(0.25). BCEWithLogitsLoss was used for the output layer. For the final training, the model trained for 250 epochs and used ensemble over 5 random initializations. The final model had 220 parameters.

To determine the best configurations, RepeatedStratifiedKFold CV was used for 15 folds. During these training steps, early stopping was utilized.

Config	Architecture	CV F1 (macro)	CV AUC-ROC	Avg epoch
k12_16	12→16→1	**0.758 ± 0.085**	0.706	7.9
k8_16	8→16→1	0.745 ± 0.068	0.722	6.1
k8_32_8	8→32→8→1	0.744 ± 0.069	0.705	9.0

Figure 6: Result of config selection StratifiedKFold cross validation

Based on these results, the best configuration and the one used was k12_16: 12 input features, 16 neurons in the hidden layer, 0.25 dropout (randomly zero 25% of neurons, helps prevent overfitting), weight decay of 1e-12, and learning rate of 1e-3.

2.3 Results

2.3.1 Development and Training Time

The development time for the Part II solutions were significantly longer than for the Part I solutions. This is mostly due to the large amount of experimentation that was needed to find a model that performed well with the, frankly, pretty hard to use dataset.

The machine used to train the Part II models used an NVIDIA GeForce RTX 4060 Ti and was able to make use of CUDA acceleration. The training time for the final traditional model was 20.6 s. The training time for the final, smaller neural net was 268 s (5-seed ensemble).

2.3.2 Numerical Metrics

- **Traditional:**

Metric	CV estimate	Training (full dev set)
Accuracy	–	0.7848
F1 (macro)	0.724 ± 0.066	0.6815
F1 (minority class=1)	–	0.5000
AUC-ROC	0.738	0.7562
Threshold	0.626	–
Training time	–	20.6 s

Figure 7: Numerical results of the final traditional model

- **Deep Learning:**

Metric	CV estimate	Training (full dev set)
Accuracy	–	0.7658
F1 (macro)	0.758 ± 0.085	0.6823
F1 (minority class=1)	–	0.5195
AUC-ROC	0.706	0.7904
Threshold	0.496	–
Training time	–	268 s (5-seed ensemble)

Figure 8: Numerical results of the final deep learning model

2.3.3 Confusion Matrix

Two sets of confusion matrices were generated for this part of the project. The main model was trained on the full training set due to the dataset being so small and Professor Beichel

having access to the test set. To assess generalization, a version of the model was re-fit to 80% of the training data and "tested" on the remaining 20% holdout dev data.

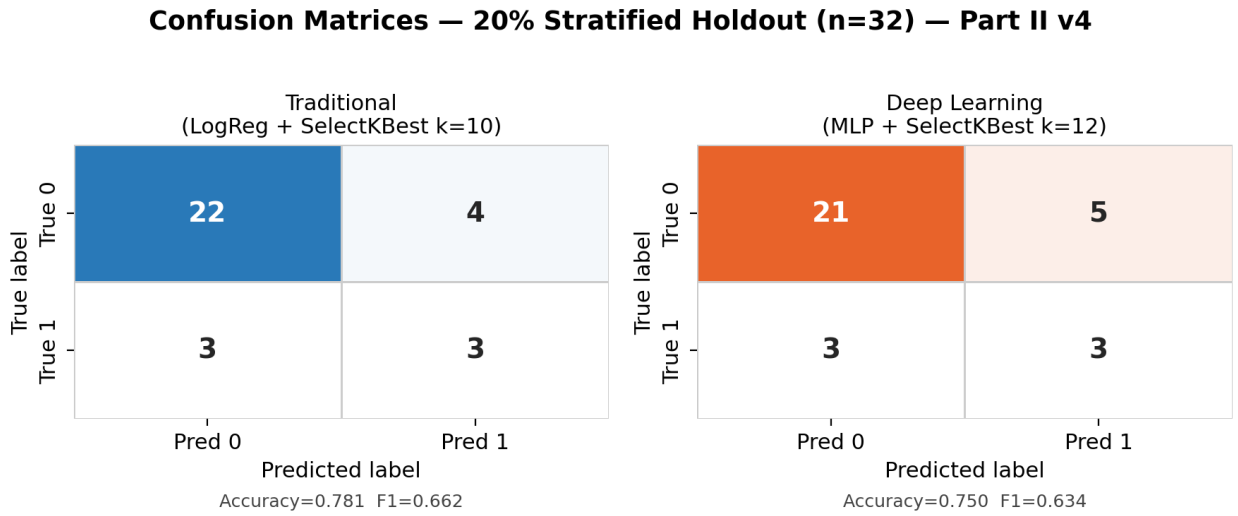

Figure 9: Confusion matrices for Part II models fit to 80% of the training data. Shows model generalization

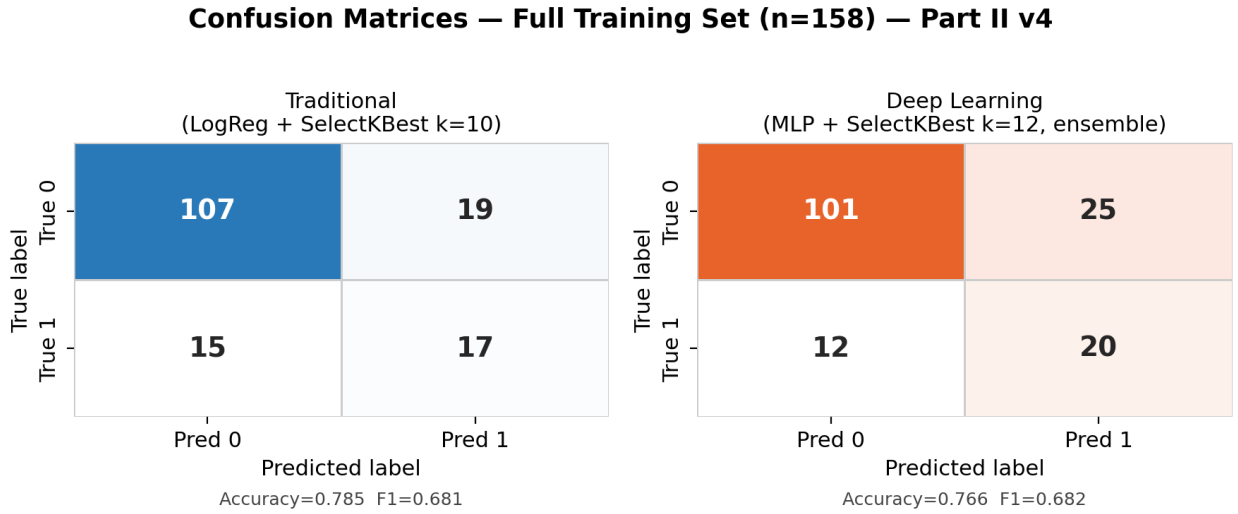

Figure 10: Confusion matrices for both Part II models, not re-fit

Comparing the full models to the refitted models and the held-out 20%, the accuracy and F1 scores are similar. This indicates to me that the model does a good job of generalizing and is not overfitting to the training set too much. For all CMs, the traditional model and deep learning model share the same most common error (false class 1). This makes sense given the very small number of positive samples in the dataset.

2.3.4 Predicted Results on Test Set

Based on the results of testing the models on the training set, I believe it will continue to perform well on the test set. What will influence it's performance going forward is mostly the class balance of the test set; if it's way different, there may be degradation in the F1 score. Due to the flexible nature of MLPs, I think there is potential for it to perform better on the test set than on the training data as well as better than the traditional model.

One note is that while the F1 score for the deep learning model is slightly higher than the traditional model, the AUC-ROC is slightly lower. This could mean the deep learning model is highly optimized for the given threshold (0.5) to balance precision and recall, but it ranks positive/negative examples poorly across all potential thresholds. This often suggests the model is good at making a firm classification decision but is less robust overall.

2.4 Conclusion

2.4.1 Model Comparison

The following figures were generated based on both Part II models:

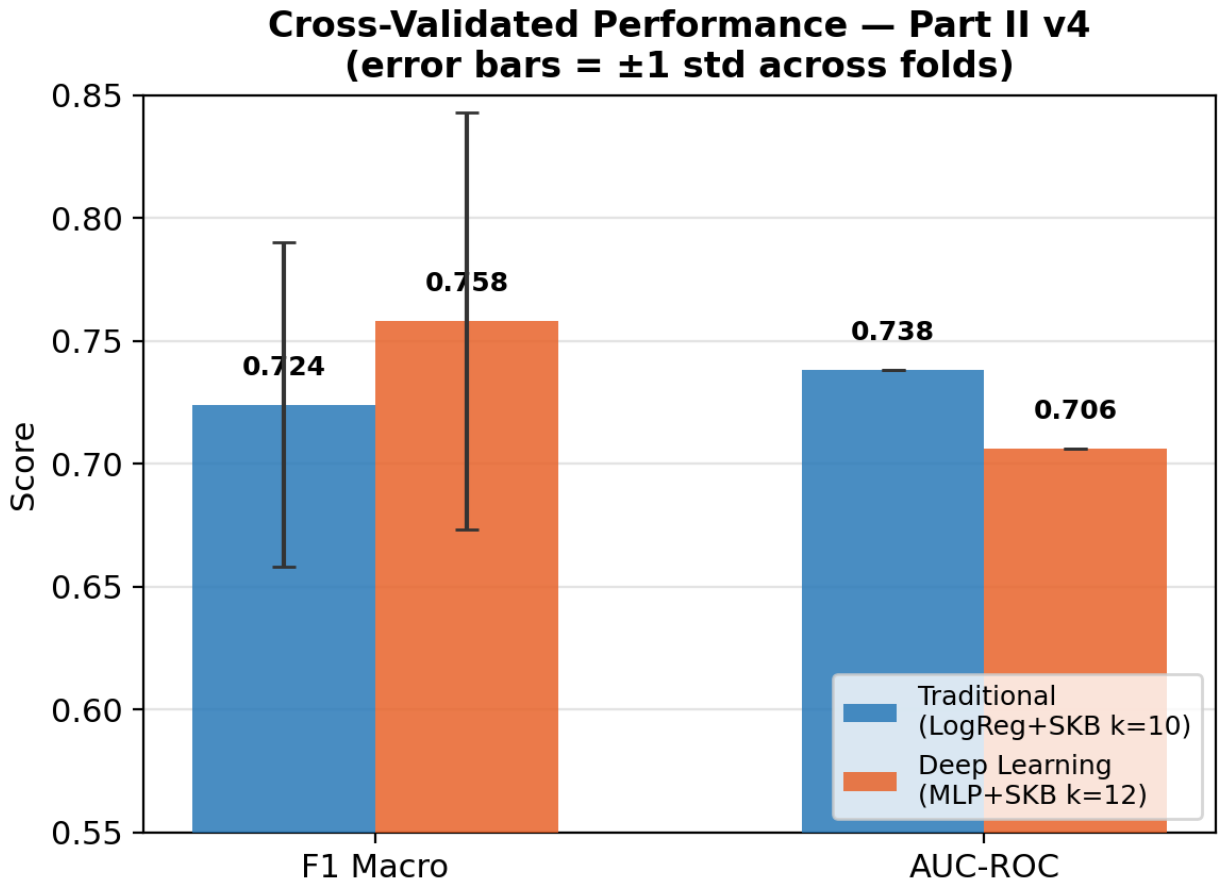

Figure 11: Comparison of CV results for macro F1 and AUC-ROC for Part II models

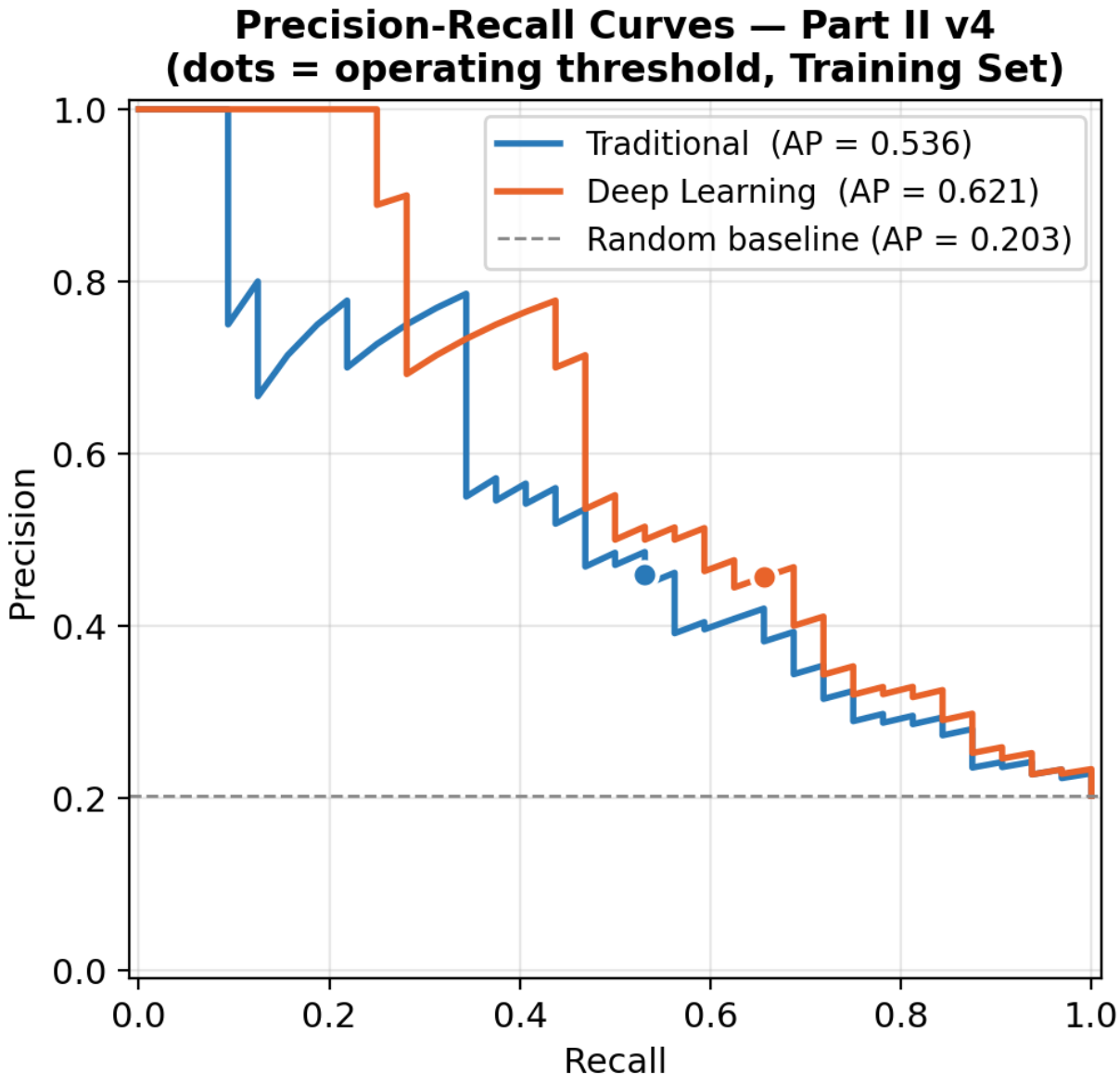

Figure 12: Comparison of precision-recall results for both Part II models

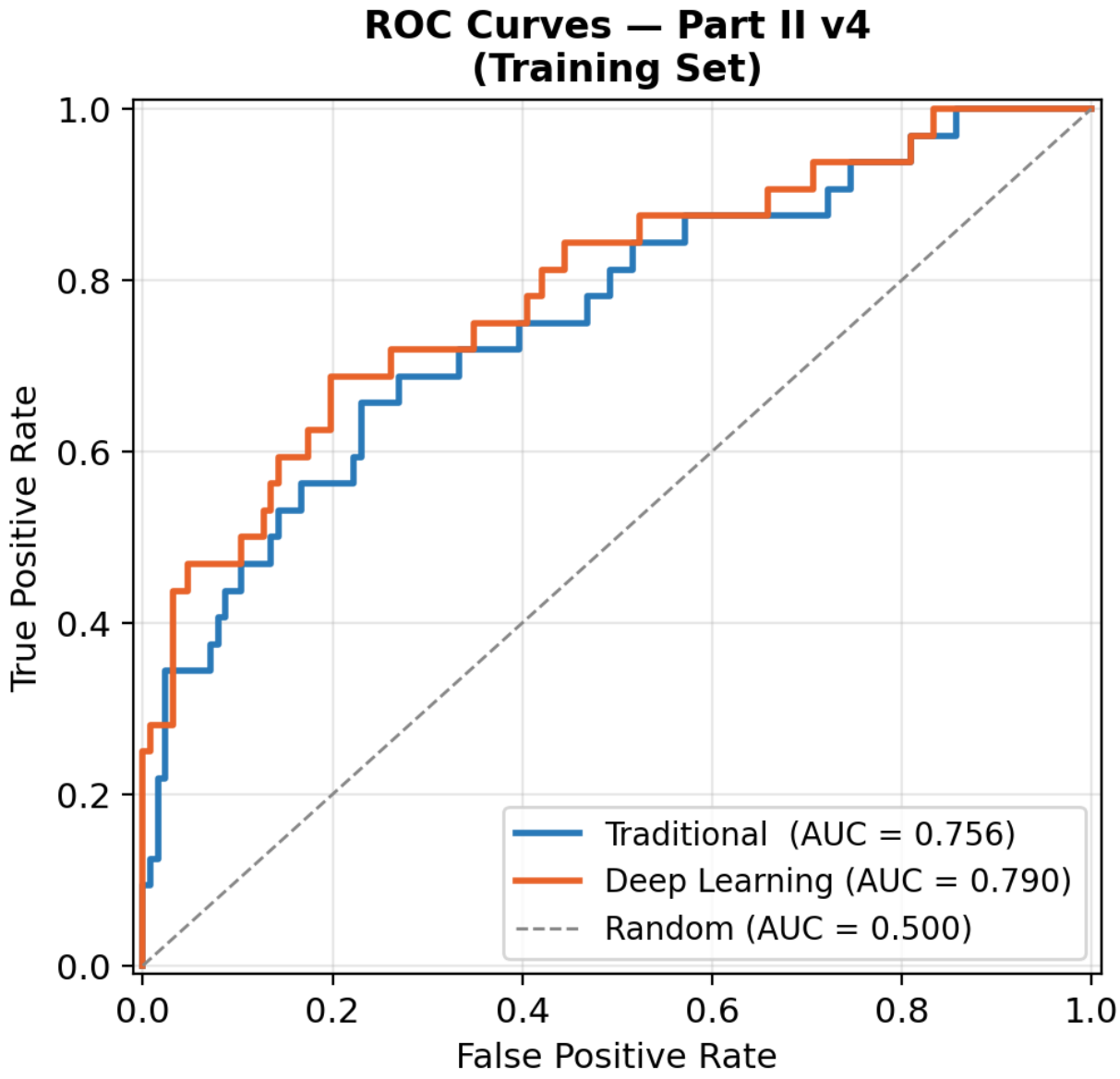

Figure 13: Comparison of ROC curves for both Part II models

The deep learning model generally performed slightly better, showing a higher macro F1 score than the traditional model. However, it did have larger standard deviation, and its AUC-ROC score was slightly lower as mentioned in the section above. If the dataset had more samples, I think the deep learning model would have out-performed the traditional model in every aspect.

2.4.2 Lessons Learned

This part of the project was an exercise in dealing with sub-optimal data. The low amount of samples made it very difficult to get performance out of the models. I also thought, based on what we learned in class, that synthetic minority class generation may help, but that's

not a universal solution!

I chose to develop both models even though only one was required because I wanted more practice developing deep learning models.